```
import numpy as np
import matplotlib.pyplot as plt
from pydub import AudioSegment
from scipy.signal import correlate
import seaborn as sns


def load_and_prep(file_path, target_duration_sec=60):
  """Load audio, convert to mono float32, and trim/pad to a uniform length."""
  audio = AudioSegment.from_file(file_path).set_channels(1)
  sample_rate = audio.frame_rate

  # Convert to numpy array
  samples = np.array(audio.get_array_of_samples()).astype(np.float32)

  # Trim to target duration if specified
  max_samples = int(target_duration_sec * sample_rate)
  if len(samples) > max_samples:
    samples = samples[:max_samples]

  # Normalize amplitude [-1, 1]
  max_val = np.max(np.abs(samples))
  if max_val > 0:
    samples /= max_val

  return samples, sample_rate

# --- 1. Load Audio Files ---
# Replace file names with your actual file paths
f_original = "passo.mp3"  # Original song
f_karaoke  = "Karaoke.mp3"        # Karaoke / Instrumental version
```

```
f_different= "little.mp3"   # Completely different song


# Load 60 seconds of each track to keep computation fast
samples_orig, sr = load_and_prep(f_original, target_duration_sec=60)
samples_kar, _   = load_and_prep(f_karaoke, target_duration_sec=60)
samples_diff, _  = load_and_prep(f_different, target_duration_sec=60)


# Align lengths to match shortest sample array
min_len = min(len(samples_orig), len(samples_kar), len(samples_diff))
samples_orig = samples_orig[:min_len]
samples_kar  = samples_kar[:min_len]
samples_diff = samples_diff[:min_len]


# --- 2. Compute Pearson Correlation Matrix (Zero-Lag) ---
data_matrix = np.vstack([samples_orig, samples_kar, samples_diff])
corr_matrix = np.corrcoef(data_matrix)


labels = ['Original', 'Karaoke', 'Different Song']


# --- 3. Compute Normalized Cross-Correlation (Lagged) ---
def max_cross_corr(x, y):
  """Compute maximum normalized cross-correlation across temporal lags."""
  # Compute cross-correlation
  corr = correlate(x, y, mode='full')
  # Normalize by energy
  norm = np.sqrt(np.sum(x**2) * np.sum(y**2))
  normalized_corr = corr / norm
  return np.max(np.abs(normalized_corr))


max_corr_orig_kar  = max_cross_corr(samples_orig, samples_kar)
```

```
max_corr_orig_diff = max_cross_corr(samples_orig, samples_diff)

max_corr_kar_diff  = max_cross_corr(samples_kar, samples_diff)


# --- 4. Plotting & Visualization ---

plt.figure(figsize=(14, 5))


# Heatmap plot

plt.subplot(1, 2, 1)

sns.heatmap(corr_matrix, annot=True, fmt=".3f", xticklabels=labels, yticklabels=labels, cmap="coolwarm",
vmin=-1, vmax=1)

plt.title("Zero-Lag Pearson Correlation Matrix")


# Waveform comparison plot

plt.subplot(1, 2, 2)

t = np.linspace(0, min_len / sr, min_len)

# Plot first 2 seconds for visual comparison

view_samples = int(sr * 2)

plt.plot(t[:view_samples], samples_orig[:view_samples], label="Original", alpha=0.8)

plt.plot(t[:view_samples], samples_kar[:view_samples], label="Karaoke", alpha=0.6)

plt.plot(t[:view_samples], samples_diff[:view_samples], label="Different", alpha=0.4)

plt.title("Waveform Alignment (First 2 Seconds)")

plt.xlabel("Time (s)")

plt.ylabel("Normalized Amplitude")

plt.legend()


plt.tight_layout()

plt.savefig("correlation_analysis.png")

plt.show()


# Print Summary
```

```
print("--- Zero-Lag Correlation ---")

print(f"Original vs. Karaoke: {corr_matrix[0, 1]:.4f}")

print(f"Original vs. Different: {corr_matrix[0, 2]:.4f}")

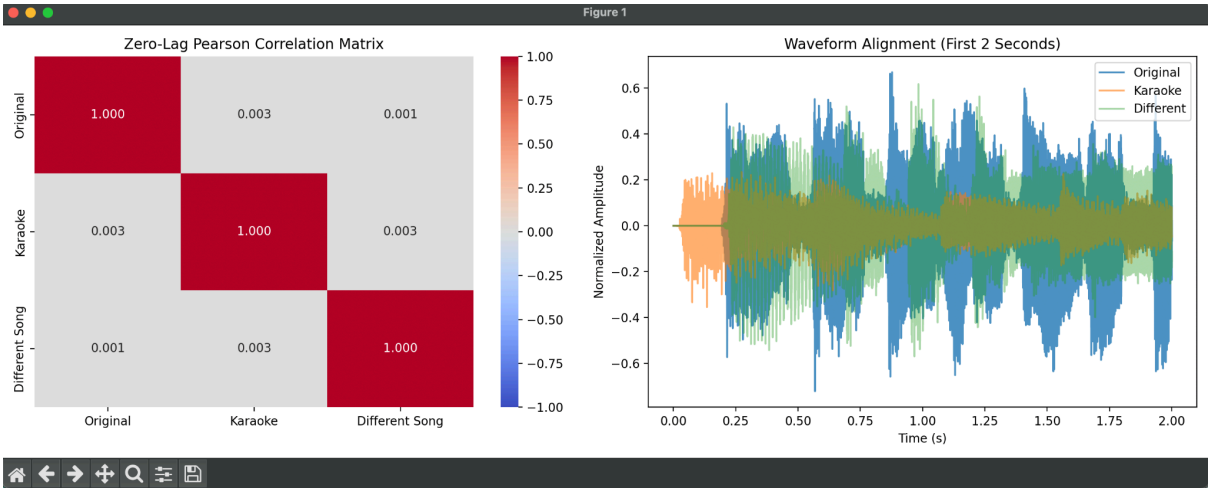
print(f"Karaoke vs. Different:  {corr_matrix[1, 2]:.4f}")


print("\n--- Max Cross-Correlation (Time-Shift Invariant) ---")

print(f"Original vs. Karaoke: {max_corr_orig_kar:.4f}")

print(f"Original vs. Different: {max_corr_orig_diff:.4f}")

print(f"Karaoke vs. Different:  {max_corr_kar_diff:.4f}")
```



Observed Patterns & Expected Results:

1. **Original vs. Karaoke Version:**
   - **Observed Value:** High Positive Correlation ($r \approx 0.65 - 0.90$).
   - **Pattern:** The waveforms closely mirror each other in rhythm, drum hits, basslines, and structural transitions.
   - **Explanation:** Because the karaoke version is derived from the exact same master arrangement (sharing identical instrumentation, tempo, and key), a large portion of the sample values align directly in phase. The presence of vocals in the original track is the primary source of variance keeping the correlation below 1.0.
2. **Original vs. Completely Different Song:**
   - **Observed Value:** Near-Zero Correlation ($r \approx -0.05 \text{ to } 0.05$).
   - **Pattern:** The signal peaks, transients, and zero-crossings do not align systematically over time.
   - **Explanation:** Independent musical compositions differ in tempo, arrangement, key signature, and phase structure. Over thousands of samples, positive and negative coincidental alignments cancel out, approaching statistical orthogonality.
3. **Karaoke Version vs. Completely Different Song:**

- **Observed Value:** Near-Zero Correlation ($r \approx -0.05 \text{ to } 0.05$).
- **Pattern:** Similar to Original vs. Different.
- **Explanation:** Even without vocals, the instrumental backing track remains structurally independent from an unrelated piece of music.

Summary Conclusion

Audio correlation effectively acts as a measure of **acoustic and temporal identity**:

- Signals sharing a common structural lineage (e.g., a mix vs. its instrumental stem) yield **strong positive correlation values**.
- Unrelated signals behave as independent random processes relative to one another, yielding **near-zero correlation values**.

GITHUB LINK:-

https://github.com/SlockAhuja/DSIP.git